Translated from Indonesian to English - www.onlinedoctranslator.com

JOB SHEET

POLYMORPHISM

1. Competence

After conducting the experiment on this jobsheet, it is hoped that students will be able to:

a. Understand the concept and basic forms of polymorphism

b. Understand the concept of virtual method invocation

c. Applying polymorphism to the creation of heterogeneous collections

d. Applying polymorphism to method parameters/arguments

e. Apply object casting to change the shape of the object

2. Introduction

Polymorphism is the ability of an object to take multiple forms. The most common use of polymorphism in OOP occurs when a superclass reference is used to refer to an object of a subclass. In other words, when an object is declared in a superclass, it can be instantiated as an object of the subclass. From this description, it can be seen that the concept of polymorphism can be applied to classes that have an inheritance relationship (generalization relationship or IS-A).

In addition to classes with inheritance relationships, polymorphism can also be applied to interfaces. When an object is declared for an interface, it can be used to reference objects from classes that implement that interface.

To illustrate the above description, the following example is given. There is an interface**Vegetarian**, and super class**Animal**. Then a class is created **Deer**which is a sub-class of**Animal**and implements to**Vegetarian**. While class **Lion**sub-class of**Animal**, and does not implement to**Vegetarian**.

```
public interfaceVegetarian{}
public classAnimal{}
public classDeerextendsAnimalimplementsVegetarian{} public classLion
extendsAnimal{}
```

From the class declaration above, the example object declaration below will show which is valid and which is invalid based on the concept of polymorphism.

```
eerd=new        eer();
ionl=new        ion();

animala = d;
animala2 = l;
vegetarianv      d;
```

valid

```
vegetarianv2 = l;
```

invalid

From the example above,**a**(type**Animal**) can be used to refer to objects**d**(is the object of**Deer**), because class**Deer**is a derivative of**Animal**. As well**a2**(type **Animal**), can also be used to reference objects**l**(is the object of**Lion**), because class**Lion** is a derivative of**Animal**. Object**v**(declared from interface **Vegetarian**) can also be used to refer to objects**d**(objects of class**Deer**), this can be done because the class**Deer**implements to interface **Vegetarian**.

While the object**v2**(decorated from the interface**Vegetarian**)**Can't** used to reference objects**l**(objects of class**Lion**), because class**Lion does not implement** to the interface**Vegetarian**. The illustration can be
shows the basic concepts and forms of polymorphism.

Virtual method invocation

Virtual method invocation occurs when an overriding method is called from a polymorphic object. It's called virtual because the method recognized by the compiler and the method executed by the JVM are different.

```
public classAnimal{
        public voidwalk(){
                System.out.println("The animal is walking around the jungle");

        }
```

```
}

public classDeerextendsAnimal{
        @override
        public voidwalk(){
                System.out.println("The deer is walking around the jungle");

}
```

When there is a polymorphic object**a**, for example:

```
Deerd=newDeer(); Animala
= d;
```

Then the overriding method is called from it, then at that time a virtual method call occurs, such as:

```
a.walk();
```

At compile time, the compiler will recognize the method**walk()**what will be called is the method**walk()**what's in the class**Animal**, because the object**a** type **Animal**. But when it is executed (run time), what the JVM executes is the method **walk()**what's in the class**Deer**. It would be different if the walk() method was called from an object.**d**(not a polymorphic object), such as

```
d.walk();
```

then method**walk()**which is recognized at compile time by the compiler and which executed at runtime by the JVM are both methods**walk()**which exists in class**Deer**(because of the object**d**declared from class**Deer**).

Heterogeneous Collection

With the concept of polymorphism, array variables can be made heterogeneous. This means that the array can contain a variety of different objects. For example:

```
Animalanimal[] =newAnimal[2];
animalArray[0] =newDeer();
animalArray[1] =newLion();
```

From the example the first data from the array**animalArray**contains objects**Deer**, and the second data from**animalArray**contains objects**Lion**. This can be done

because of the array**animalArray**declared from class**Animal**(superclass of **Deer**And**Lion**).

Polymorphic Argument

Polymorphism can also be applied to a method's arguments. The goal is to allow the method to accept argument values from various object types. For example, a new class is created as follows:

```
public classHuman{
        public voiddrive(Animal anim){
                anim.walk();
        }
}
```

Pay attention to the method**drive()**, he has an argument in the form of**Animal**. Because **Animal**have subclasses**Lion**And**Deer**, then the method**drive()**will be able to accept arguments in the form of objects from**Deer**and objects from**Lion**.

```
Deerd=newDeer();
Lionl=newLion(); Humanhum =
newHuman(); hum.drive(d);

hum.drive(l);
```

instanceof operator

The instanceof operator can be used to check whether an object is an instance of a particular class. The result of instanceof is a Boolean value. Suppose we create objects d and l.

```
Deerd=newDeer();
Lionl=newLion(); Animala1
= d;
Animala2 = l;
```

If then the instanceof operator is used, for example

a1instanceofDeera Awill produce true a2instanceofLiona Awill produce true

Object Casting

Object casting is used to change the type of an object. If an object from a subclass is changed to its superclass, this is considered upcasting. For example:

```
Deerd=newDeer();
Animala1 = d;//This process can also be called upcasting
```

A downcast occurs when an object of a superclass is transformed into an object of a subclass. For example:

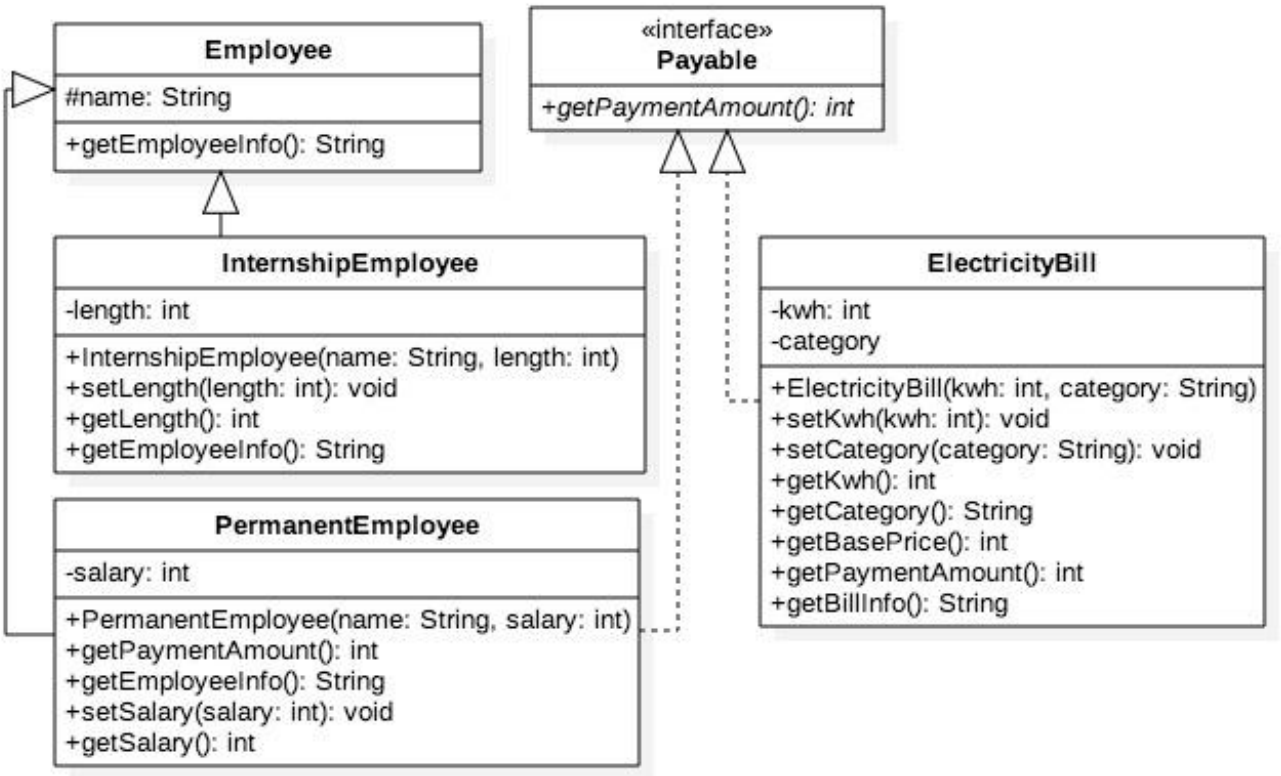
```
eerd=newDeer();
Animala1 = d;//This process can also be called upcasting
eerd = (Deer) a1; //downcasting process
```

The downcasting process is often referred to as explicit casting, because of the form The purpose of the casting must be written in brackets, in front of the object being cast. will be cast.

3. Case Study

For the experiment on this jobsheet, the class diagram below will be used.

This:



In a company, the owner has to pay salaries every month.

permanent employees and electricity bills. In addition to permanent employees, the company also have interns, where these employees do not receive a salary.

4. Experiment 1 – Basic forms of polymorphism

4.1. Experimental Steps

1. Create a class**Employee**

```
public class Employee {
    protected String name;

    public String getEmployeeInfo(){
        return "Name = "+name;
    }
}
```

2. Create an interface**Payable**

```
public interface Payable {
    public int getPaymentAmount();
}
```

3. Create a class**Internship Employee**, a subclass of**Employee**

```
public class InternshipEmployee extends Employee{
    private int length;

    public InternshipEmployee(String name, int length) {
        this.length = length;
        this.name = name;
    }
    public int getLength() {
        return length;
    }
    public void setLength(int length) {
        this.length = length;
    }
    @Override
    public String getEmployeeInfo(){
        String info = super.getEmployeeInfo()+"\n";
        info += "Registered as internship employee for "+length+" month/s\n";
        return info;
    }
}
```

4. Create a class**Permanent Employee**, a subclass of**Employee**And implements to**Payable**

```
3   public class PermanentEmployee extends Employee implements Payable{
4       private int salary;
5
6       public PermanentEmployee(String name, int salary) {
7           this.name = name;
8           this.salary = salary;
9       }
10      public int getSalary() {
11          return salary;
12      }
13      public void setSalary(int salary) {
14          this.salary = salary;
15      }
16      @Override
        public int getPaymentAmount() {
18          return (int) (salary+0.05*salary);
19      }
20      @Override
        public String getEmployeeInfo(){
22          String info = super.getEmployeeInfo()+"\n";
23          info += "Registered as permanent employee with salary "+salary+"\n";
24          return info;
25      }
26  }
```

5. Create a class**Electricity Bill**which implements the interface **Payable**

```
3   public class ElectricityBill implements Payable{
4       private int kwh;
5       private String category;
6
7       public ElectricityBill(int kwh, String category) {
8           this.kwh = kwh;
9           this.category = category;
10      }
11      public int getKwh() {
12          return kwh;
13      }
14      public void setKwh(int kwh) {
15          this.kwh = kwh;
16      }
17      public String getCategory() {
18          return category;
19      }
20      public void setCategory(String category) {
21          this.category = category;
22      }
23      @Override
25      public int getPaymentAmount() {
26          return kwh*getBasePrice();
27      }
28      public int getBasePrice(){
29          int bPrice = 0;
30          switch(category){
31              case "R-1" : bPrice = 100;break;
32              case "R-2" : bPrice = 200;break;
33          }
34          return bPrice;
35      }
36      public String getBillInfo(){
37          return "kWH = "+kwh+"\n"+
38                  "Category = "+category+"("+getBasePrice()+" per kWH)\n";
39      }
   }
```

6. Create a class**Tester1**

```
3   public class Tester1 {
4       public static void main(String[] args) {
5           PermanentEmployee pEmp = new PermanentEmployee("Dedik", 500);
6           InternshipEmployee iEmp = new InternshipEmployee("Sunarto", 5);
7           ElectricityBill eBill = new ElectricityBill(5, "A-1");
8           Employee e;
9           Payable p;
10          e = pEmp;
11          e = iEmp;
12          p = pEmp;
13          p = eBill;
14      }
15  }
```

4.2. Questions

1. What classes are derived from the class**Employee**?
2. What classes implement the interface?**Payable**?
3. Pay attention to class**Tester1**, lines 10 and 11. Why**e**, can be filled with objects**pEmp**(is an object of class**Permanent Employee**) and objects **iEmp**(is an object of class **Internship Employee**) ?

4. Pay attention to class**Tester1**, lines 12 and 13. Why**p**, can be filled with objects**pEmp** (is an object of class **Permanent Employee**) and objects**eBill**(is an object of class **Electricity Bill**) ?
5. Try adding the syntax:

   p = iEmp;
   e = eBill;

   on lines 14 and 15 (the last line in the method**play**) ! What causes the error?
6. Draw conclusions about the basic concepts/forms of polymorphism!

5. Experiment 2 – Virtual method invocation

5.1. Experimental Steps

1. In this experiment, the classes and interfaces used in the previous experiment will still be used.
2. Create a new class with name**Tester2**.

```
3    public class Tester2 {
4        public static void main(String[] args) {
5            PermanentEmployee pEmp = new PermanentEmployee("Dedik", 500);
6            Employee e;
7            e = pEmp;
8            System.out.println(""+e.getEmployeeInfo());
9            System.out.println("------------------");
10           System.out.println(""+pEmp.getEmployeeInfo());
11       }
12   }
```

3. Run the class**Tester2**, and the following results will be obtained:

```
run:
Name = Dedik
Registered as permanent employee with salary 500

------------------
Name = Dedik
Registered as permanent employee with salary 500
```

5.2. Questions

1. Pay attention to class**Tester2**above, why the summons **e.getEmployeeInfo()** on line 8 And **pEmp.getEmployeeInfo()** on line 10 produces the same result?
2. Why call the method**e.getEmployeeInfo()**is called a virtual method invocation, whereas **pEmp.getEmployeeInfo()**No?

3. So what is meant by virtual method invocation? Why? called virtual?

6. Experiment 3 – Heterogeneous Collection

6.1. Experimental Steps

1. In this 3rd experiment, the classes and interfaces from the previous experiment will still be used.
2. Create a new class**Tester3**.

```
3   public class Tester3 {
4       public static void main(String[] args) {
5           PermanentEmployee pEmp = new PermanentEmployee("Dedik", 500);
6           InternshipEmployee iEmp = new InternshipEmployee("Sunarto", 5);
7           ElectricityBill eBill = new ElectricityBill(5, "A-1");
            Employee e[] = {pEmp, iEmp};
            Payable p[] = {pEmp, eBill};
            Employee e2[] = {pEmp, iEmp, eBill};
11      }
12  }
```

6.2. Questions

1. Pay attention to the array**e**on the 8th line, why can it be filled with objects of different types, namely objects**pEmp**(object of **Permanent Employee**) and objects**iEmp**(object of **Internship Employee**) ?

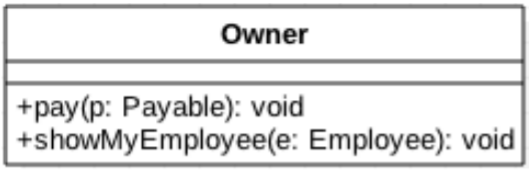
2. Also pay attention to line 9, why the array**p**also filled with objects of different types, namely objects**pEmp**(object of **Permanent Employee**) and objects**eBill**(object of **Electricity Billing**) ?

3. Pay attention to line 10, why does an error occur?

7. Experiment 4 – Polymorphism, instanceof and object casting arguments

7.1. Experimental Steps

1. Experiment 4 also still uses the classes and interfaces used in the previous experiment.
4. Create a new class with name**Owner**.**Owner**can make payments to both permanent employees and electricity bills via the method**pay()**. In addition, you can also display information about permanent employees and interns using the method **showMyEmployee()**.




```
3   public class Owner {
4       public void pay(Payable p){
5           System.out.println("Total payment = "+p.getPaymentAmount());
6           if(p instanceof ElectricityBill){
7               ElectricityBill eb = (ElectricityBill) p;
8               System.out.println(""+eb.getBillInfo());
9           }else if(p instanceof PermanentEmployee){
10              PermanentEmployee pe = (PermanentEmployee) p;
11              pe.getEmployeeInfo();
12              System.out.println(""+pe.getEmployeeInfo());
13          }
14      }
15      public void showMyEmployee(Employee e){
16          System.out.println(""+e.getEmployeeInfo());
17          if(e instanceof PermanentEmployee)
18              System.out.println("You have to pay her/him monthly!!!");
19          else
20              System.out.println("No need to pay him/her :)");
21      }
22  }
```

2. Create a new class**Tester4**.

```
3   public class Tester4 {
4       public static void main(String[] args) {
5           Owner ow = new Owner();
6           ElectricityBill eBill = new ElectricityBill(5, "R-1");
7           ow.pay(eBill);//pay for electricity bill
8           System.out.println("--------------------------------");
9
10          PermanentEmployee pEmp = new PermanentEmployee("Dedik", 500);
11          ow.pay(pEmp);//pay for permanent employee
12          System.out.println("--------------------------------");
13
14          InternshipEmployee iEmp = new InternshipEmployee("Sunarto", 5);
15          ow.showMyEmployee(pEmp);//show permanent employee info
16          System.out.println("--------------------------------");
17          ow.showMyEmployee(iEmp);//show internship employee info
18      }
19  }
```

3. Run class**Tester4**, and the following results will be obtained:

```
run:
Total payment = 1000
kWH = 5
Category = R-1(200 per kWH)

--------------------------------
Total payment = 525
Name = Dedik
Registered as permanent employee with salary 500

--------------------------------
Name = Dedik
Registered as permanent employee with salary 500

You have to pay her/him monthly!!!
--------------------------------
Name = Sunarto
Registered as internship employee for 5 month/s

No need to pay him/her :)
```

7.2. Questions

1. Pay attention to class**Tester4**line 7 and line 11, why the call**ow.pay(eBill)** And**ow.pay(pEmp)**can be done, even if you pay attention to the method **pay()**that is in the class**Owner** have arguments/parameters type **Payable**?

If you pay attention more eBill details are objects of pEmp **Electricity Bill** And which are objects of **Permanent Employee**?

2. So what is the purpose of making type arguments?**Payable**on method **pay()**that is in the class**Owner**?
3. Try on the last line of the method**main()**that is in the class **Tester4** added command**ow.pay(iEmp);**

```
3   public class Tester4 {
4       public static void main(String[] args) {
5           Owner ow = new Owner();
6           ElectricityBill eBill = new ElectricityBill(5, "R-1");
7           ow.pay(eBill);//pay for electricity bill
8           System.out.println("-----------------------------");
9
10          PermanentEmployee pEmp = new PermanentEmployee("Dedik", 500);
11          ow.pay(pEmp);//pay for permanent employee
12          System.out.println("-----------------------------");
13
14          InternshipEmployee iEmp = new InternshipEmployee("Sunarto", 5);
15          ow.showMyEmployee(pEmp);//show permanent employee info
16          System.out.println("-----------------------------");
17          ow.showMyEmployee(iEmp);//show internship employee info
18
            ow.pay(iEmp);
20      }
21  }
```

Why does the error occur?

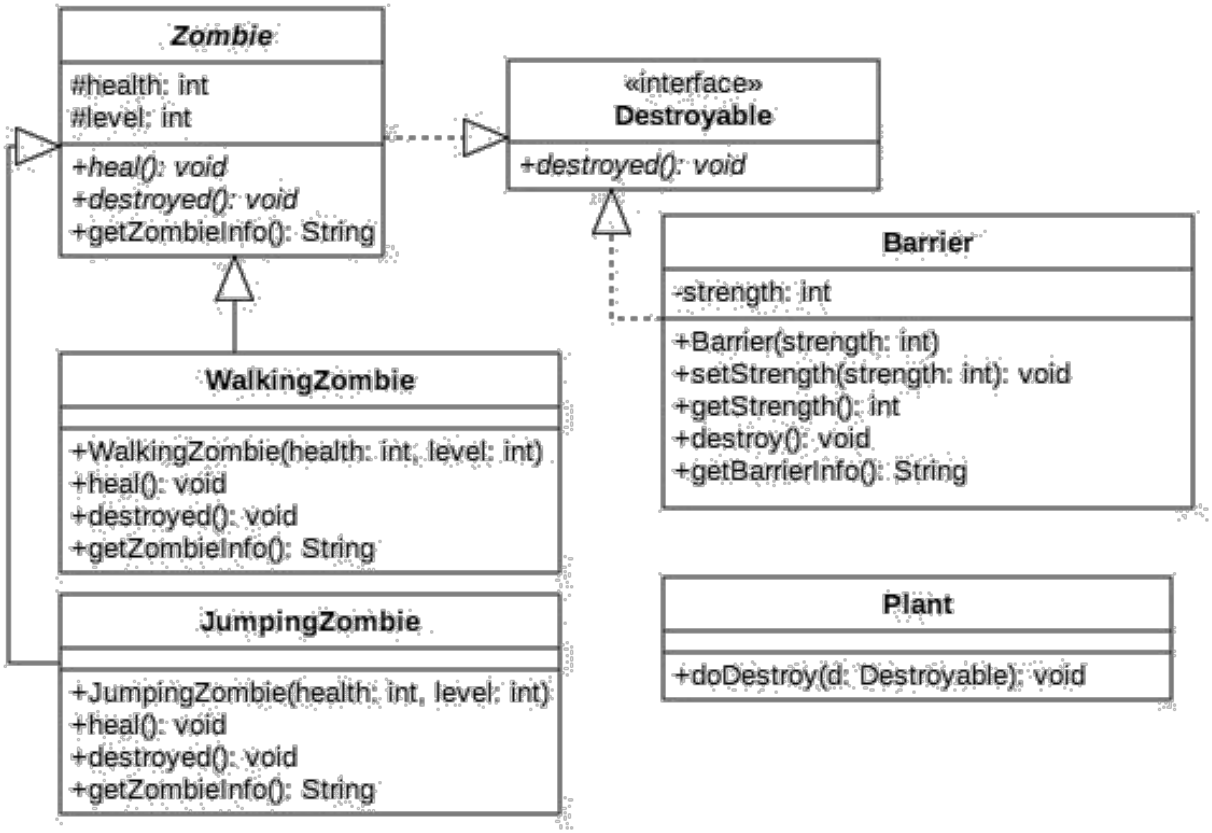
4. Pay attention to class**Owner**, is required for what syntax**p instanceof****ElectricityBill**on the 6th line?
5. Look again at the Owner class in line 7, for what purpose is object casting there (**ElectricityBill eb = (ElectricityBill) p**) necessary? Why is the object**p**of the type**Payable**must be cast to in the object**eb**of the type**Electricity Bill?**

8. Tasks

In the game, Zombies and Barriers can be destroyed by Plants and can heal themselves. There are two types of Zombies: Walking Zombies and Jumping Zombies. These two Zombies have different healing methods, as well as different ways of destroying them, which are determined by the following rules:

- On Walking Zombie
    - Healing: Healing is determined based on the level of the zombie in question.
        - §§If the zombie is level 1, then every time it heals, health will increase by 210%
        - §§If the zombie is level 2, then every time it heals, health will increase by 30%
        - §§If the zombie is level 3, then every time it heals, health will increase by 40%
    - Destruction: every time it is destroyed, health will decrease. 2%
- On Jumping Zombie o
    - Healing: Healing is determined based on the level of the zombie in question.
        - §§If the zombie is level 1, then every time it heals, health will increase by 30%
        - §§If the zombie is level 2, then every time it heals, health will increase by 40%
        - §§If the zombie is level 3, then every time it heals, health will increase by 50%
    - Destruction: every time it is destroyed, health will decrease by 1%

Create a program from the class diagram below!



Example: if the Tester class is like below:

```
3   public class Tester {
4       public static void main(String[] args) {
5           WalkingZombie wz = new WalkingZombie(100, 1);
6           JumpingZombie jz = new JumpingZombie(100, 2);
7           Barrier b = new Barrier(100);
8           Plant p = new Plant();
9           System.out.println(""+wz.getZombieInfo());
10          System.out.println(""+jz.getZombieInfo());
11          System.out.println(""+b.getBarrierInfo());
12          System.out.println("----------------------");
13          for(int i=0;i<4;i++){//Destroy the enemies 4 times
14              p.doDestroy(wz);
15              p.doDestroy(jz);
16              p.doDestroy(b);
17          }
18          System.out.println(""+wz.getZombieInfo());
19          System.out.println(""+jz.getZombieInfo());
20          System.out.println(""+b.getBarrierInfo());
21      }
22  }
```

Will produce the output:

